Project ESCAPE: Structural Discovery via Entropy in BLT Across Code and Natural Language

Anonymous ACL submission

Abstract

We investigate whether entropy-triggered boundaries in the Byte Latent Transformer (BLT) align with syntactic constituent starts beyond a density-matched null. Project ESCAPE compares Python, C++, and English prose. We restore the patcher's 512-byte sliding window, extract entropies for all 16,300 files, freeze the matching tolerance on a calibration split, and score the held-out split with one-to-one matching against 10,000 permutations of each file's own patch lengths. Held-out start F1 exceeds the null in all three languages (Python 0.152 vs. 0.032; C++ 0.165 vs. 0.036; English 0.495 vs. 0.147). Against the strongest structure-blind baseline (indentation or word starts), however, BLT underperforms it in all three languages. By constituent type, forced-opener constructs such as `if` and `def` align more strongly than open-ended ones, contrary to our prediction. Under injected code noise, alignment degrades only slightly, while noise lowers bytes per patch. Entropy is a surface statistic, and none of these results is evidence of semantic understanding.

1 Introduction

Byte-level models remove a fixed subword vocabulary, but computation still needs to be allocated across long input sequences. BLT groups bytes into variable-length patches using a small next-byte entropy model (Pagnoni et al., 2025). A boundary at the start of a syntactic constituent could indicate that this allocation tracks structure. Rare identifiers, word starts, and formatting can also induce boundaries, so proximity alone cannot distinguish these explanations.

ESCAPE asks whether entropy-triggered boundaries align with constituent starts more than a null with the same patch-length distribution and boundary count (P1). We also ask whether they align more at starts than at ends (P2), more in code than in prose (P3), and less for constructs whose opening is forced than for open-ended ones (P4). A further question is whether any extra patching near memory-unsafe C++ persists after identifier controls. These are empirical hypotheses: entropy bounds branching ($H \leq \log B$) but is neither a grammatical branching factor nor a measure of program semantics. This report gives held-out results for P1–P4 against both the null and structure-blind baselines, span IoU as a secondary metric, and robustness to code noise (Objective 4).

This submission contributes (i) a byte-exact extraction pipeline for the BLT patcher that restores the reference 512-token sliding window missing from the public Hugging Face conversion, (ii) a calibration-first scoring protocol whose null preserves each file's patch-length multiset and boundary count and whose structure-blind baselines make line and word segmentation the burden of proof, and (iii) held-out measurements across three languages with robustness results, all reproducible from retained per-byte artifacts. Section 3 fixes the protocol; Section 4 describes data and checkpoint; Section 5 reports results.

2 Background and Related Work

BLT dynamically segments byte sequences so that expensive latent computation operates over patches (Pagnoni et al., 2025). Our analysis concerns its entropy patcher, not downstream generation accuracy.

Subword tokenization and its failure modes. Standard pretraining front ends fix a vocabulary before the model sees data. BPE learns a merge table over a corpus and then applies it uniformly (Sennrich et al., 2016). The resulting segmenta-

tion is known to align poorly with morphology, to segment rare or multilingual tokens unstably, and to spend the same number of units on boilerplate as on subtle logic, because the compression ratio is frozen in advance. Interpretability work on tokenizers has therefore concentrated on the vocabularies themselves; whether a learned segmentation corresponds to anything independently verifiable is rarely asked, because a static merge table is not a claim a model makes.

Tokenization-free models. ByT5 removes the vocabulary but not the uniformity: every byte is processed with equal compute (Xue et al., 2022), and CANINE substitutes learned character hashing with the same uniform treatment (Clark et al., 2022). Neither architecture decides, at inference time, where a boundary belongs. BLT is the first architecture at scale in which segmentation is both byte-level and dynamic: each boundary is a claim the model makes about its own next-byte uncertainty, which is precisely why it can be checked against external structure.

Structural probing. It is established that trained representations encode syntax: structural probes recover parse trees from contextual embeddings (Hewitt and Manning, 2019), and code pretraining objectives yield embeddings from which program structure is recoverable (Feng et al., 2020). These results, however, require either a probe trained for the purpose or a model trained with code-specific objectives. None of this literature asks the question ESCAPE asks: whether an unsupervised, compression-only signal with no parser in the loop recovers constituent structure as a side effect. The BLT paper reports efficiency and downstream accuracy but never inspects whether the induced patches line up with structure that can be verified independently; that is the gap this work addresses, and code is the natural test bed because its parse trees provide machine-checkable ground truth.

Statistical segmentation precedents. Using local predictability to decide where units begin is an old idea outside neural models: unsupervised word segmentation infers boundaries from statistical cues such as prediction strength and transition probabilities (Brent, 1999; Goldwater et al., 2009). BLT moves this intuition inside a byte-level neural model, but the induced boundaries have not been checked against independently verifiable structure — which is precisely the check the segmentation literature has always had, since word boundaries are directly observable. ESCAPE applies the same check to the patcher: the AST is never seen by the model and exists only as evaluation ground truth.

We use code AST starts and predicted English constituent starts as external structural references. Python samples come from CodeSearchNet (Husain et al., 2019), and C++ uses a subset of The Stack (Kocetkov et al., 2022). English paragraphs come from WikiText-103 (Merity et al., 2017) and are parsed with benepar (Kitaev and Klein, 2018). These reference structures differ in granularity and reliability, motivating language-specific calibration and reporting.

3 Evaluation Protocol

Byte contract and metric. All positions are frozen UTF-8 byte offsets; spans are half-open. Versioned JSONL interfaces carry sample metadata and hashes, patcher outputs, ground-truth nodes, and region annotations. Validators check joins, hashes, lengths, patch tiling, and entropy coverage. Entropy at byte i is the model's global Shannon entropy

$$H_i = -\sum_{v \in \mathcal{V}} p_\theta(v \mid x_{<i}) \log p_\theta(v \mid x_{<i}) \qquad (1)$$

over its 260 output symbols, in nats. Predictions are interior boundaries labelled `entropy`. Targets are unique starts of eligible, non-wrapper, positive-length nodes, excluding byte zero and file end. Maximum-cardinality one-to-one matching pairs predictions and targets within k bytes. With T matches, P predictions and G targets, precision is T/P, recall T/G and F1 $2T/(P+G)$, pooled across files. Ends are scored the same way.

Frozen tolerance and null. Each language is split at the file level by a seeded SHA256 rank: 20% calibration, 80% test. Only calibration files evaluate $k \in \{0, 1, 2, 4, 8\}$, and we choose the k maximizing the median per-file F1 margin over the null, ties to the smaller k. The hashed record is verified at scoring, and there is no test-time override. For each file, H_0 shuffles the (length, termination) pairs of all patches except the last, which stays in place so that file end never becomes a scored boundary, and rebuilds the endpoints. This preserves file length, the patch-length multiset and the number of entropy boundaries. Each of 10,000

seeded permutations shuffles every test file independently. We report the null mean, the one-sided permutation value, and paired file-bootstrap 95% intervals (2,000 replicates).

Fixing the final patch matters more than it looks: if the last patch were shuffled too, file end would become a candidate boundary in some resamples, and a file whose final bytes are a closing delimiter or punctuation would systematically penalize the null, inflating BLT's margin. The fixed-endpoint construction removes that artefact by construction, and because the multiset of lengths is otherwise preserved exactly, the null is the strongest density-matched baseline available from the file's own outputs.

The calibration curve is monotone and decisive: the median per-file F1 margin over the null shrinks as k grows, from 0.121 to 0.010 in Python, 0.126 to 0.014 in C++ and 0.357 to 0.002 in English across the grid $\{0, 1, 2, 4, 8\}$, so the criterion selects $k = 0$ for all three languages. Selecting the strictest tolerance is a statement about the data, not a choice to hide behind: larger tolerances would close the gap between BLT and the null precisely because the null's density, not its placement, is what a loose tolerance rewards.

Structure-blind baselines. Under the same targets, k and matching, we score boundaries after every newline, at indentation changes, and at every word start. A structural reading requires BLT to beat these, since most code constituents begin after a line break and most prose constituents begin at a word. We report BLT minus baseline F1 with a paired file bootstrap.

Matching and depth stratification. One-to-one matching prevents a single prediction from crediting two nested targets; with any-match counting, nested starts at nearby offsets could inflate recall, and the two conventions would diverge for $k > 0$. Alignment is additionally computed per raw tree-sitter depth stratum, with recall as the per-stratum metric because all strata share the same prediction set, so precision differences are definitional rather than empirical. Raw depth is not comparable across languages: Python and C++ carry very different amounts of grammar scaffolding per statement, and the benepar constituent trees are collapsed differently again. Cross-language depth normalization is therefore left to the final submission; per-depth curves here are within-language evidence about where in the tree boundaries concentrate.

Replication artefacts. Two implementation-level effects are measured rather than assumed away. First, the patcher processes byte streams in 8,192-token chunks under the restored window, so files longer than 8,191 bytes have positions whose context is truncated at a chunk edge; the BPP summary records these positions per file, and they are reported as part of the run metadata. Second, the entropy threshold is compared strictly in float64 against the stored float32 entropies, so every trigger decision is reproducible from the saved arrays alone, which the cross-process reproducibility check exploits.

Worked example. The patcher's segmentation of the first lines of a short Python function, with patch starts marked by `|`:

```
d|e|f |f|i|b|onacci|(|n|):
|if n |< |2|:
|return |n|
|return |fibonacci(n |- 1) |+
|fibonacci(n - 2)
```

Boundaries fall at statement starts after indentation (`if`, `return`), at function arguments, and at nearly every byte inside the first occurrence of an identifier, while a token already committed stays whole on reuse (`fibonacci(n` in the final line is a single patch). The same qualitative pattern holds in C++ — boundaries open statements after delimiters and indentation, identifiers split at first use and stay whole afterwards, and boilerplate stretches produce long patches, which is why C++ files carry the largest within-file BPP variance. The example shows all three sources of entropy at once: structure, lexical novelty, and formatting — which is why the baselines of Section 3 are needed to separate them.

Provenance. Reproducibility is enforced mechanically rather than by convention. The frozen run directory carries per-file SHA-256 sums; the scoring configuration, seeds and file identities are recorded with every table; and the report's numbers are generated by a build script that reads the retained artifacts and refuses synthetic inputs. The corpus manifest pins every file's bytes by hash, and the calibration record stores the calibration file identities and the selected tolerance, so a from-scratch re-run either reproduces the reported numbers or fails loudly.

Gate	What it checks	Outcome
G1	patcher weights load, counts and checksums match provenance	pass
G2	tokenizer emits BOS + byte offsets (EOS expectation was a mis-specified criterion)	amended
G3	HF port logits equal an independent re-implementation (max $\lvert\Delta\rvert$ 1.05×10^{-4})	pass
G4	entropy is Shannon entropy in nats (float64 recomputation matches)	pass
G5	boundary rule equals HF's own `patch_lengths` on 28/28 inputs	pass
G6	context: influence vanishes beyond the 512-token window (reference regime)	pass
G7	behavior: low entropy in repeated tails, high boundary density on random bytes	amended
G8	determinism: identical reruns are bit-identical	pass

Table 1: Checkpoint-fidelity gate battery. The two amended gates are criterion corrections (EOS handling and an uncalibrated random-byte threshold), not pipeline fixes; the declared outcomes remain on record.

4 Data and Checkpoint

The frozen manifest has 16,300 samples: 8,000 Python functions (CodeSearchNet), 8,000 C++ files (The Stack, smol) and 300 English paragraphs (WikiText-103). All bytes were restored and verified against stored hashes. Calibration used 1,600/1,600/60 files and testing 6,400/6,400/240 (Python/C++/English). The selected tolerances were $k = 0$, 0 and 0.

We use the Hugging Face conversion `itazap/blt-1b-hf` (revision `91aa6b8e`), loading only its 99.5M-parameter patcher. It runs in float32 with threshold 1.3354 nats, no length cap, and the reference 512-token sliding window and 8,192-token chunks, which the port omits. Extraction ran on four RTX 2080 Ti GPUs of the IIIT-H Ada cluster, and all 16,300 files completed with consistent artifacts. Six of eight original fidelity gates passed; two failed (tokenizer EOS and a random-byte threshold). The amended checks, which use the actual BOS-plus-byte input and require over 90% random-byte boundary density, pass. Rerun on the cluster's software stack, the amended gate set passed. This establishes agreement with an independent reference implementation, not identity with the original weights.

Every corpus byte was restored and verified against the stored sha256 before inference, and derived parses were built only from verified bytes. Tree-sitter error recovery triggers on 28.1% of C++ files (unresolved macro and header context); these files are retained and flagged rather than dropped, because recovery is localized and excluding them would bias the sample toward simpler files. The confound that motivates the structure-blind baselines is measured, not assumed: 31.8% of Python and 22.5% of C++ constituent starts sit exactly at an indentation-change byte, so a proposer that splits at line or indentation boundaries would score well without knowing anything about grammar. Saved outputs are reproducible: cross-process reruns of the same bytes agree to within 8.3×10^{-6} nats with identical boundary sets, so every downstream table can be recomputed from the retained per-byte arrays. Files longer than 8,191 bytes cross an 8,192-token chunk boundary, after which the patcher's context resets; these bytes are identified per file in the run metadata and constitute a measured replication artefact rather than an unmodeled one. Inference throughput is about 3,400 tokens per second per GPU in float32 at 1.36 GiB peak memory, so the full corpus is a few GPU-hours rather than a training run; no model training occurs at any stage.

5 Results

P1: alignment beyond the null. Table 2 gives held-out start alignment. The F1 margin over H_0 is +0.120 [0.117, 0.122] in Python, +0.129 [0.125, 0.134] in C++ and +0.348 [0.341, 0.354] in English. Precision/recall are 0.083/0.822, 0.095/0.646 and 0.342/0.894. A sufficient-but-not-necessary mechanism predicts precision above recall. Precision is below recall in all three languages. BLT proposes many more boundaries than there are targets, so most boundaries fall away from constituent starts.

The precision–recall asymmetry has a mechanical reading under the patcher. A boundary certifies only that the local next-byte distribution was flat enough to cross τ; because $H \leq \log B$, every entropy-triggered boundary sits where at least $e^{\tau} \approx 3.8$ continuations were live. Identifier interiors, strings and comments are flat without being syntactic, so predictions vastly outnumber targets and precision is structurally capped, while the same density guarantees high recall against the comparatively sparse target set. The asymmetry itself is therefore expected on entropy grounds; what the proposal's mechanism predicted — that cheap interiors and forced closing delimiters would push precision above recall — is not supported. On magnitude: a margin of roughly 0.12–0.13 F1 against a null that already shares the file's own patch-length distribution is a

Language	k	Test files	P	R	F1	H_0 F1	ΔF1 [95% CI]	p
Python	0	6,400	0.083	0.822	0.152	0.032	+0.120 [0.117, 0.122]	≤1e-4
C++	0	6,400	0.095	0.646	0.165	0.036	+0.129 [0.125, 0.134]	≤1e-4
English	0	240	0.342	0.894	0.495	0.147	+0.348 [0.341, 0.354]	≤1e-4

Table 2: Held-out entropy-boundary alignment with constituent starts (P1). k was frozen on the calibration split. H_0 F1 is the permutation mean of the fixed-EOF density-matched null (10,000 permutations); intervals are paired file bootstraps; p is the one-sided permutation value.

Lang.	BLT	NL	Indent	Word	BLT−best [95% CI]
Python	0.152	0.000	0.394	0.245	-0.242 [-0.247, -0.237]
C++	0.165	0.041	0.281	0.339	-0.173 [-0.181, -0.167]
English	0.495	0.000	0.008	0.769	-0.274 [-0.280, -0.268]

Table 3: Start F1 of structure-blind baselines under the same targets, k and matching: boundary after every newline (NL), at indentation changes, and at every word start. The last column compares BLT with the strongest baseline for that language.

Lang.	Type	Opener	Starts	R	H_0 R	ΔR
Python	`assignment`	open	39	0.872	0.156	+0.715
Python	`binary_operator`	open	7,276	0.683	0.184	+0.498
Python	`call`	open	33,643	0.766	0.168	+0.598
Python	`expression_statement`	open	43,709	0.864	0.169	+0.695
Python	`for_statement`	det.	2,829	0.934	0.162	+0.772
Python	`function_definition`	det.	485	0.891	0.166	+0.724
Python	`if_statement`	det.	10,084	0.903	0.158	+0.745
Python	`return_statement`	open	7,010	0.798	0.249	+0.550
Python	`while_statement`	det.	263	0.954	0.161	+0.794
C++	`assignment_expression`	open	8,078	0.452	0.133	+0.319
C++	`binary_expression`	open	285,064	0.506	0.144	+0.362
C++	`call_expression`	open	456,491	0.595	0.138	+0.457
C++	`declaration`	open	249,479	0.773	0.142	+0.632
C++	`expression_statement`	open	486,954	0.655	0.138	+0.517
C++	`for_statement`	det.	21,326	0.823	0.136	+0.686
C++	`function_definition`	det.	101,613	0.768	0.134	+0.635
C++	`if_statement`	det.	126,003	0.795	0.146	+0.649
C++	`return_statement`	open	93,611	0.562	0.144	+0.418
C++	`while_statement`	det.	4,102	0.942	0.150	+0.792

Table 4: Start recall by constituent type (P4). Types are rematched separately, so rows are not additive. A start shared by several nodes is credited to the outermost one, which is why nested types such as Python `assignment` have few starts. Opener class follows the empirically checked `taxonomy.json`.

strong density-independent effect, but it is smaller than the 0.24–0.27 gap by which structure-blind segmentation beats BLT, which is the comparison the structural claim must survive.

Baselines. The strongest structure-blind proposer is the indentation baseline for Python (F1 0.394), the word-start baseline for C++ (0.339) and the word-start baseline for English (0.769). BLT underperforms it in all three languages (Python -0.242 [-0.247, -0.237]; C++ -0.173 [-0.181, -0.167]; English -0.274 [-0.280, -0.268]). With the frozen $k = 0$, the newline baseline scores near zero: an indented statement starts after its indentation, not at the byte after the newline, and each English sample is a single paragraph. A margin over H_0 that does not survive this comparison indicates line or word segmentation rather than syntactic structure. The scale of the confound is known before scoring: 31.8% of Python and 22.5% of C++ constituent starts sit exactly at an indentation-change byte, and English constituent starts largely coincide with word starts, which is why the indentation and word baselines dominate in their respective languages.

P2 and P3. The start margin exceeds the end margin in all three languages (end margins -0.017, -0.035 and -0.079 for Python, C++ and English). These two margins share predictions and are compared descriptively. Pooled by domain, the code margin (+0.129) is smaller than the prose margin (+0.348), so P3 is not supported; in English, BLT also falls short of the word-start baseline (Table 3). Mean bytes per patch are 5.68, 7.86 and 3.77 (Python, C++, English), with mean within-file BPP variances 69.4, 527.8 and 10.3. The large C++ variance is interpretable: C++ files mix long boilerplate stretches (headers, declarations, brace blocks) with dense logic, so within a single file the patcher alternates between long cheap patches and short uncertain ones, whereas prose paragraphs are comparatively homogeneous. The code–prose contrast therefore goes the opposite way from the proposal's prediction: prose, not code, produces the largest margin over the null.

P4 and depth. Averaged over types, the recall margin over H_0 for forced-opener constructs is +0.759 in Python and +0.691 in C++, against +0.611 and +0.451 for open-ended ones (Table 4). The forced-opener margin is larger in both code languages, so the data do not support P4 as stated. One reading is that a start boundary precedes the keyword, where the choice of statement is still open; the forcing we predicted applies to the bytes after it, which starts do not measure. Figure 1 shows the start recall margin by tree depth. The margin is large at every reported depth in both languages — 0.43–0.71 in C++ and 0.51–0.71 in

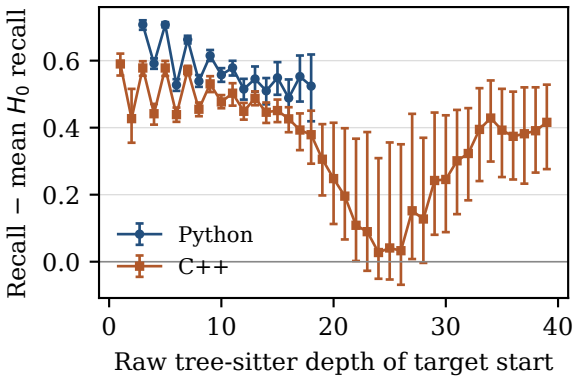


Figure 1: Held-out start recall minus mean H_0 recall by raw tree-sitter depth, for strata with at least 200 target starts. Recall is used because all strata share one prediction set. Raw depth is not comparable across languages. Whiskers are file-bootstrap 95% intervals of recall.

Lang.	Files	BLT	H_0	Line	Word	BLT$-H_0$ [95% CI]
Python	6,400	0.351	0.342	0.546	0.342	+0.008 [0.007, 0.010]
C++	6,188	0.372	0.356	0.606	0.409	+0.016 [0.013, 0.018]
English	240	0.377	0.334	0.049	0.397	+0.042 [0.040, 0.045]

Table 5: Secondary metric: mean best single-patch IoU over unique constituent spans, held-out split (files with at least one span). H_0 averages 100 fixed-EOF permutations; Line and Word are newline and word-start segmentations.

Python through depth 15 — and declines only slowly toward the leaves, so boundaries track constituents throughout the tree rather than only at statement level. Raw depth is not comparable across languages (grammar scaffolding differs), which is why depth curves are within-language evidence here.

Secondary: span IoU. For each unique constituent span we take its best IoU with a single patch (Table 5). BLT exceeds the null by +0.008 [0.007, 0.010] in Python, +0.016 [0.013, 0.018] in C++ and +0.042 [0.040, 0.045] in English. In code, line segmentation matches spans far better (0.546 and 0.606). Word segmentation also matches spans at least as well as BLT in C++ and English (0.409 vs. 0.372 and 0.397 vs. 0.377), another instance of the segmentation confound measured on the secondary metric. Patches are much shorter than most constituents, so IoU understates alignment by design; it is secondary to start P/R.

Objective 4: robustness to code noise. On 1,000 held-out files per language we inject the four noise types of our proposal, each on its own: missing statement semicolons (C++), a one-space indentation change on 20% of lines, consistent camelCase↔snake_case renaming of 50% of convertible identifiers, and adjacent-letter swaps in 20% of keywords. Targets are the clean constituents mapped to their noised offsets, so we measure whether boundaries still track the intended structure. The margin over H_0 is +0.115 (Python) and +0.131 (C++) on clean code (Table 6). The largest drop comes from indentation noise in Python (-0.016 [-0.018, -0.015]) and indentation noise in C++ (-0.013 [-0.015, -0.012]); every noised margin stays at or above +0.099 and +0.118. Patching thus degrades gracefully rather than failing. Keyword typos change BPP most (-0.23 and -0.34 bytes per patch), since misspelled keywords are unpredictable and receive more patches. Renaming identifiers costs almost nothing, which is consistent with the precision analysis above: identifiers are already near-arbitrary for the patcher, so renaming them does not redistribute much entropy.

Objective 5: status. The memory-unsafe analysis is reported as work in progress. Its design is fixed: region spans for dereference, address-of, `new` and `delete` are extracted with tree-sitter queries, and per-region boundary density and inverse bytes-per-patch are regressed on a memory-unsafe indicator together with identifier-entropy controls on the held-out C++ split. The design matrices and region annotations are computed and retained with the run, but the inferential reading is deferred to the final submission pending a label audit. The framing matters: a pointer token is short, frequent and lexically predictable, so any effect is expected to flow through surface unpredictability rather than semantics.

Objectives summary. P1 (alignment beyond the null) is supported in all three languages; P2 (starts over ends) is supported with negative end margins; the structure-blind comparison, P3 and P4 as originally stated are not supported; O4 (graceful degradation under noise) is supported. The pattern is consistent with entropy being a genuine but shallow structural signal: the patcher tracks constituent starts far beyond chance, but most of what it tracks is line and word segmentation, and the parts of the hypothesis that required entropy to distinguish grammatical choice from lexical novelty did not survive the held-out test.

Lang.	Noise	Edits	ΔF1 vs. H_0	Change vs. clean [95% CI]	BPP	Change vs. clean [95% CI]
Python	clean	0	+0.115	–	5.61	–
Python	camel↔snake	4,601	+0.114	-0.001 [-0.002, -0.001]	5.56	-0.05 [-0.06, -0.04]
Python	indentation	4,416	+0.099	-0.016 [-0.018, -0.015]	5.61	+0.00 [-0.00, 0.01]
Python	keyword typo	1,626	+0.112	-0.003 [-0.004, -0.003]	5.38	-0.23 [-0.25, -0.21]
C++	clean	0	+0.131	–	7.88	–
C++	camel↔snake	83,037	+0.129	-0.002 [-0.003, -0.002]	7.78	-0.10 [-0.11, -0.09]
C++	indentation	68,746	+0.118	-0.013 [-0.015, -0.012]	7.75	-0.13 [-0.14, -0.12]
C++	keyword typo	17,884	+0.131	-0.000 [-0.001, 0.000]	7.54	-0.34 [-0.36, -0.31]
C++	missing `;`	25,972	+0.129	-0.002 [-0.002, -0.002]	7.82	-0.06 [-0.06, -0.05]

Table 6: Objective 4: robustness to code noise on the same held-out files per language. Each noise type is applied alone; targets are the clean constituents mapped to noised offsets; k is the frozen value. The null uses 1,000 fixed-EOF permutations per file; intervals are paired file bootstraps of noised minus clean.

6 Remaining Work and Conclusion

BLT's entropy boundaries sit at constituent starts far more often than its own patch lengths would by chance. Relative to the strongest structure-blind baseline, however, BLT underperforms it in all three languages. Any structural claim must therefore be made relative to line and word segmentation, not the null alone. The measurement pipeline, the frozen calibration and the held-out scores reported here are fixed inputs to the remaining work; the final submission extends the analysis rather than re-running it. The concrete plan, keyed to the 31 October final deadline:

Dates (2026)	Work
3–10 Oct	Review the memory-unsafe-region regression (O5): audit the region labels (`*`, `&`, `new`, `delete`) against tree-sitter queries, run the identifier-entropy conditioning, and report whether any residual survives the surface-predictability confound.
10–17 Oct	Test the P4 reversal against byte positions immediately after the opening keyword, where the forcing was predicted to apply; implement cross-language depth normalization (O3) and re-score the per-depth analysis with it.
17–24 Oct	Sweep Objective 4 noise rates per type; freeze the statistical amendments confirmed from the mid-submission reviewer feedback and re-run scoring once with the final protocol.
24–31 Oct	Final ablations and figures; write-up to 8 pages excluding references; code freeze, packaging (`Trimax-Final.zip`) and viva presentation.

Table 7: Plan for the remainder of the project. The first three rows are analysis that consumes the frozen held-out scores; only the protocol amendments in the third row can change reported numbers, and they will be re-validated on the calibration split before any test split is rescored.

Limitations

The checkpoint is a third-party conversion, and the sliding window was restored by us. The restoration is validated against an independent reference implementation of the same weights — agreement is architectural and behavioral (logit agreement to 1.05×10^{-4}, exact boundary agreement) rather than bitwise with the original FAIR checkpoint, which is gated. English targets come from predicted benepar parses, so the English ground truth carries the parser's error rate, and the English paragraphs were inspected in an earlier exploratory analysis before this protocol was frozen. About 28% of C++ files trigger parser error recovery; they are included and flagged, which preserves representativeness at the cost of noisier targets for those files. Python samples are functions that may share repositories, so file bootstraps need not generalize across projects; the cluster bootstrap treats files as the independent unit and would underestimate variance if within-project correlation matters. The null fixes the final patch so that file end never becomes a scored boundary; word and newline baselines are deterministic and not density matched, so they are stronger comparisons than a density-equalized alternative would be. Objective 4 uses one noise rate per type and 1,000 null permutations per file, and span IoU uses 100. Files longer than 8,191 bytes have chunk-reset positions whose context is truncated; these are reported per file in the run metadata.

Ethical Considerations

The project analyzes existing text and source code without executing corpus programs. Dataset provenance and licenses accompany restored inputs; CodeSearchNet carries no per-file license. This report draft and supporting implementation

used AI assistance and require author review under the course's disclosure and authorship rules.

References

Michael R. Brent. 1999. An efficient, probabilistically sound algorithm for segmentation and word discovery. *Machine Learning*, 34:71–105.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online. Association for Computational Linguistics.

Sharon Goldwater, Thomas L. Griffiths, and Mark Johnson. 2009. A bayesian framework for word segmentation: Exploring the effects of context. *Cognition*, 112(1):21–54.

John Hewitt and Christopher D. Manning. 2019. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4129–4138, Minneapolis, Minnesota. Association for Computational Linguistics.

Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.

Nikita Kitaev and Dan Klein. 2018. Constituency parsing with a self-attentive encoder. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2676–2686, Melbourne, Australia. Association for Computational Linguistics.

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srini Iyer. 2025. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, Vienna, Austria. Association for Computational Linguistics.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.